

LT 10a - Data Abstraction

Data Types

- Integer
- Float
- String
- Boolean

Data Structures

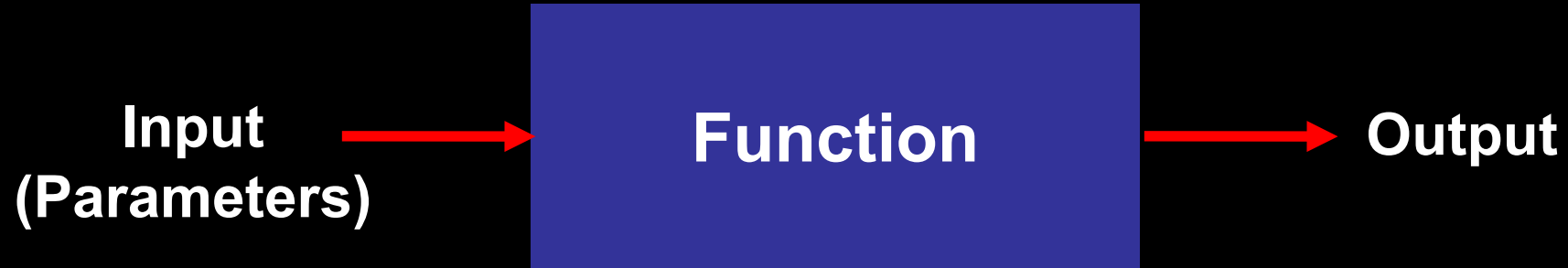
- Tuple
- List
- Dictionary

Abstraction

1. **Functional Abstraction** means defining a function by what it does, its behaviour or output, while hiding the implementation details and how it does it.

"I don't care how the function works, I just need to know what it does and how to use it."

Black Box



No need to know HOW it does the job!

Just need to know WHAT it does. 😊

Abstraction

2. **Data Abstraction** is a process of hiding the internal structure and representation of data, and only shows the operations or methods to interact with the data.

"I only need to know what is the data I'm dealing with, and don't need to know how it is being stored."

Abstract Data Type (ADT)

The definition of an ADT only mentions **the data**, without specifying how it is being stored; and the **operations** that can be performed on the data, but without specifying how they are implemented.

Example 1: A student record ADT

A **student record ADT** contains data like the name, NRIC, Class and Address of a student.

The method to create the student record ADT is known as a **Constructor**:

```
make_record(name, id, class, address)
```


Example 1: A student record ADT

The methods used to retrieve the data from the ADT are known as **Accessors** or **Getters**:

```
get_name(student)
```

```
get_class(student)
```

```
get_address(student)
```

Note: There is no method provided to retrieve the NRIC info

Example 1: A student record ADT

The methods used to change the data in the ADT are known as **Modifiers** or **Setters**:

```
set_class(student)  
set_address(student)
```

Note: A user can only modify the class and address if the NRIC info is verified to be correct.

Example 1: A student record ADT

The two **Utility methods** provided are:

```
print_info(student)
```

```
print_datatype(student)
```

Example 1: A student record ADT

Demo 1 and Demo 2

Observation from Demo 1 and 2

- Data are stored in an ADT, it doesn't matter how they are stored.
- The data in the ADT can only be accessed or modified using the methods provided, eg the Getters, Setters, Constructor and Utility methods.

Example 2: A position ADT

A **Position ADT** contains the x- and y- coordinates of the position of a point.

The **Constructor** method to create the position ADT is

```
make_point(xcoord, ycoord)
```

Example 2: A position ADT

The **Accessors** or **Getters** methods used to retrieve the coordinates are:

```
get_x(point)
```

```
get_y(point)
```

Example 2: A position ADT

The **Utility** methods are:

`midpoint(point1, point2) → Position ADT`

`gradient(point1, point2) → float`

`distance(point1, point2) → float`

`equation(point1, point2) → string  $y = mx + c$`

`display(point) → string (x, y)`

Midpoint

Point1 = (x_1, y_1) , Point2 = (x_2, y_2)

The **coordinates of the midpoint** between Point1 and Point2:

$$\text{Midpoint} = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$$

Gradient

Point1 = (x_1, y_1) , Point2 = (x_2, y_2)

The **gradient** (also called the slope) of the line joining the two points:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \quad (\text{provided } x_2 \neq x_1)$$

Distance

Point1 = (x_1, y_1) , Point2 = (x_2, y_2)

The **distance** between Point1 and Point2:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Equation

Point1 = (x_1, y_1) , Point2 = (x_2, y_2)

First, compute the **gradient** m as above.

Then, use point-slope form $y - y_1 = m(x - x_1)$, and rearrange to the form $y = mx + c$:

$$y = mx + (y_1 - mx_1)$$

So the **equation of the line** is:

$$y = mx + c \quad \text{where } c = y_1 - mx_1$$

Example 3: Rational Number ADT

A **Rational Number ADT** contains a integer numerator and a non-zero integer denominator to represent a rational number, or fraction.

The **Constructor** method to create the Rational Number ADT is `make_rat(numer, denom)`

Example 3: Rational Number ADT

The **Accessors** or **Getters** methods used to retrieve the numerator and denominator values are:

```
get_numer(rat)
```

```
get_denom(rat)
```

Example 3: Rational Number ADT

The **Utility** methods are:

`add(rat1, rat2) → Rational Number ADT`

`sub(rat1, rat2) → Rational Number ADT`

`mul(rat1, rat2) → Rational Number ADT`

`div(rat1, rat2) → Rational Number ADT`

`print_rat(rat) → string "n/d"`

Arithmetic Operations

Addition:

$$\frac{n_1}{d_1} + \frac{n_2}{d_2} = \frac{n_1 d_2 + n_2 d_1}{d_1 d_2}$$

Arithmetic Operations

Subtraction:

$$\frac{n_1}{d_1} - \frac{n_2}{d_2} = \frac{n_1 d_2 - n_2 d_1}{d_1 d_2}$$

Arithmetic Operations

Multiplication:

$$\frac{n_1}{d_1} \times \frac{n_2}{d_2} = \frac{n_1 n_2}{d_1 d_2}$$

Arithmetic Operations

Division:

$$\frac{n_1}{d_1} \div \frac{n_2}{d_2} = \frac{n_1 d_2}{d_1 n_2}$$

Example 3: Rational Number ADT

The **Predicates** methods are:

`is_equal(rat1, rat2) → Boolean`

`is_whole(rat) → Boolean`

Predicates

Equality:

$$\frac{n_1}{d_1} = \frac{n_2}{d_2} \quad \text{if and only if} \quad n_1 d_2 = n_2 d_1$$

Predicates

Whole Number:

$$\frac{n_1}{d_1} = k \quad \text{where } k \text{ is an integer, ie } n_1 \% d_1 = 0$$